

# Introduction

Organizor is the backend API for managing tenant environments. It keeps tenant records in a central catalog database and provisions a separate PostgreSQL database for each tenant.

The current implementation focuses on tenant lifecycle operations:

- creating tenant catalog records;
- onboarding tenants by creating a tenant database and applying migrations through internal operations;
- resolving tenant context for tenant-scoped requests;
- upgrading tenant databases by running migrations;
- soft-deleting tenants before hard deletion;
- exposing health checks, OpenAPI JSON, and Scalar API reference in development.

## Main Concepts

| Concept               | Description                                                                                                                                                                |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tenant                | A customer or isolated workspace tracked by <code>Id</code> , <code>Name</code> , <code>Slug</code> , <code>DatabaseName</code> , lifecycle status, and deletion metadata. |
| Catalog database      | Central PostgreSQL database that stores all tenant records.                                                                                                                |
| Tenant database       | PostgreSQL database named from the tenant slug, currently using the <code>organizor_tenant_{slug}</code> convention.                                                       |
| Tenant resolution     | Middleware step that reads <code>X-Tenant-Slug</code> , loads the tenant, and stores the current tenant in <code>ITenantContext</code> .                                   |
| Provisioning workflow | Infrastructure workflow that creates the tenant database and applies tenant migrations.                                                                                    |

## Project Layout

| Project                                        | Responsibility                                                                                                                                         |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Code7Dev.Organizor.Api</code>            | ASP.NET Core host, controllers, DTOs, mappers, middleware, OpenAPI/Scalar, health checks, CORS, and logging setup.                                     |
| <code>Code7Dev.Organizor.Application</code>    | Tenant service contracts, tenant models, onboarding, lifecycle, upgrade, and catalog-facing application logic.                                         |
| <code>Code7Dev.Organizor.Domain</code>         | Tenant entity and <code>TenantStatus</code> enum.                                                                                                      |
| <code>Code7Dev.Organizor.Infrastructure</code> | EF Core contexts, repository implementation, PostgreSQL connection string building, database creation/deletion, migration, and tenant context storage. |

# Generated Reference

DocFX builds API reference pages from the C# projects into the `api` section. Use those pages for class, interface, and method signatures, and use these articles for workflow-level documentation. The site is intended for internal developers and as an outside-IDE AI reference, so implementation details are documented where they affect behavior.

# Getting Started

## Prerequisites

- .NET SDK 10.
- PostgreSQL available to the API.
- Keycloak available to the API. The repository includes a development Keycloak compose file.
- A catalog database connection string named `Catalog`.
- PostgreSQL server settings under the `Postgres` configuration section for tenant database creation and resolution.
- Keycloak bearer-token settings under the `Keycloak` configuration section.
- DocFX installed as a .NET tool when building documentation locally.

Install DocFX if the `docfx` command is not available:

```
dotnet tool install -g docfx
```

Start the development Keycloak instance:

```
docker compose up -d
```

The compose file starts Keycloak at `http://localhost:8080`. Configure the `organizer-dev` realm, clients, roles, and users as described in [Keycloak Setup](#).

## Run the API

From the repository root:

```
dotnet run --project .\Code7Dev.Organizor.Api\Code7Dev.Organizor.Api.csproj
```

Development launch profiles expose:

| Profile | URL                                              |
|---------|--------------------------------------------------|
| http    | http://localhost:5100                            |
| https   | https://localhost:7184 and http://localhost:5100 |

On startup the API:

1. configures Serilog;
2. registers catalog and tenant database services;
3. registers application and infrastructure services;
4. seeds or migrates the catalog database;
5. maps controllers, health checks, OpenAPI, and Scalar in development.

## Development Endpoints

| Endpoint         | Purpose                                         |
|------------------|-------------------------------------------------|
| /health          | ASP.NET Core health checks.                     |
| /openapi/v1.json | OpenAPI document, available in development.     |
| /scalar          | Scalar API reference, available in development. |

Example tenant catalog request:

```
$token = "<organizer-admin JWT>"
```

```
Invoke-RestMethod `
  -Method Post `
  -Uri "http://localhost:5100/api/v1/tenants" `
  -Headers @{ Authorization = "Bearer $token" } `
  -ContentType "application/json" `
  -Body '{"name":"Acme","slug":"acme"}'
```

POST /api/v1/tenants creates a catalog record only and requires an organizer-admin token. Use POST /api/admin/tenants/onboard when you also need tenant database and Keycloak provisioning.

# Build the Documentation

From the repository root:

```
docfx .\docs\docfx.json
```

DocFX writes the generated site to `docs/_site` and generated API metadata to `docs/api`. Both folders are ignored by git and can be rebuilt.

# Authentication Flow — High-Level

The Portal UI uses Keycloak to authenticate users and determine what they are allowed to do. The process follows the standard OpenID Connect Authorization Code Flow with PKCE.

## What Happens During Login

1. The user clicks **Login**.
2. The Portal redirects the user to Keycloak.
3. Keycloak handles the login form and verifies credentials.
4. After successful login, Keycloak sends the user back to the Portal.
5. The Portal receives a secure token and uses it for API requests.

## Why Keycloak

- Centralized user management
- Role-based access control
- Secure token handling
- Standard OIDC protocol support

## What Developers Need to Know

- The Portal never handles passwords directly.
- Tokens are automatically attached to API requests.
- Protected pages require the user to be authenticated.
- Keycloak configuration determines what each user can access.

# Portal UI Structure — High-Level

The Portal UI is organized into a small set of logical building blocks that keep the application clean and maintainable.

## Main Building Blocks

### Pages

Screens the user can navigate to, such as:

- Login
- Dashboard
- Not Found

### Components

Reusable UI elements that appear across multiple pages.

### Layouts

Define the overall structure of the application, including navigation and shared UI elements.

### Services

Handle communication with the API and manage authentication state.

## Why This Structure

- Easy to extend with new features
- Clear separation between UI, logic, and layout
- Reusable components reduce duplication
- Predictable folder structure for developers

# API Integration — High-Level

The Portal UI communicates with the Organizer API to perform all business operations. Every action — from loading data to updating records — goes through the API.

## How Communication Works

- The Portal uses `HttpClient` to send requests.
- A secure access token is automatically added to each request.
- The API validates the token before processing the request.

## What Developers Should Know

- API calls are centralized through a configured `HttpClient`.
- Authentication is handled automatically.
- Errors from the API surface in the UI through standard Blazor patterns.

## Why This Matters

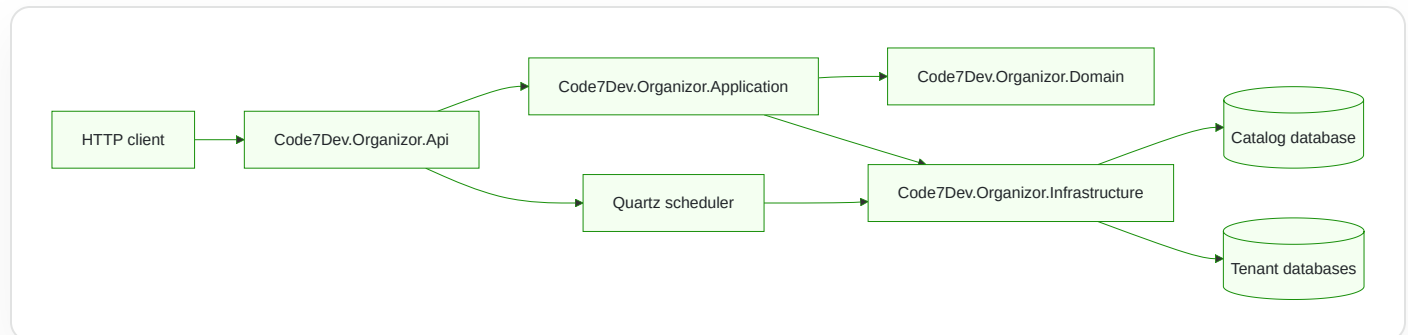
This approach ensures:

- Consistent security across the platform
- A clean separation between UI and backend logic
- The ability to scale or replace parts of the system independently



# Architecture

Organizer follows a layered structure around tenant management.



## API Layer

The API project owns transport concerns:

- controller routes for public tenant catalog operations and internal tenant operations;
- request and response DTOs;
- mapping between API DTOs and application models;
- correlation ID, exception handling, and tenant resolution middleware;
- Serilog request logging;
- CORS, health checks, OpenAPI, and Scalar setup.

The startup flow in `Program.cs` composes the application by registering catalog persistence, tenant persistence, application services, infrastructure services, and API services.

## Application Layer

The application project owns use cases and contracts. It defines interfaces for repository access, tenant database management, migration, provisioning, onboarding, upgrades, and lifecycle operations.

Current services:

| Service                 | Responsibility                                                                      |
|-------------------------|-------------------------------------------------------------------------------------|
| TenantService           | Creates catalog tenant records and reads tenants by slug or as a list.              |
| TenantOnboardingService | Creates a catalog tenant, runs provisioning, and marks the tenant active or failed. |
| TenantLifecycleService  | Soft-deletes tenants and hard-deletes already soft-deleted tenants.                 |
| TenantUpgradeService    | Runs migrations for an existing tenant database.                                    |

## Domain Layer

The domain project currently contains the tenant entity and status enum. A tenant stores identity, slug, database name, creation date, lifecycle status, provisioning failure reason, and deletion metadata.

## Infrastructure Layer

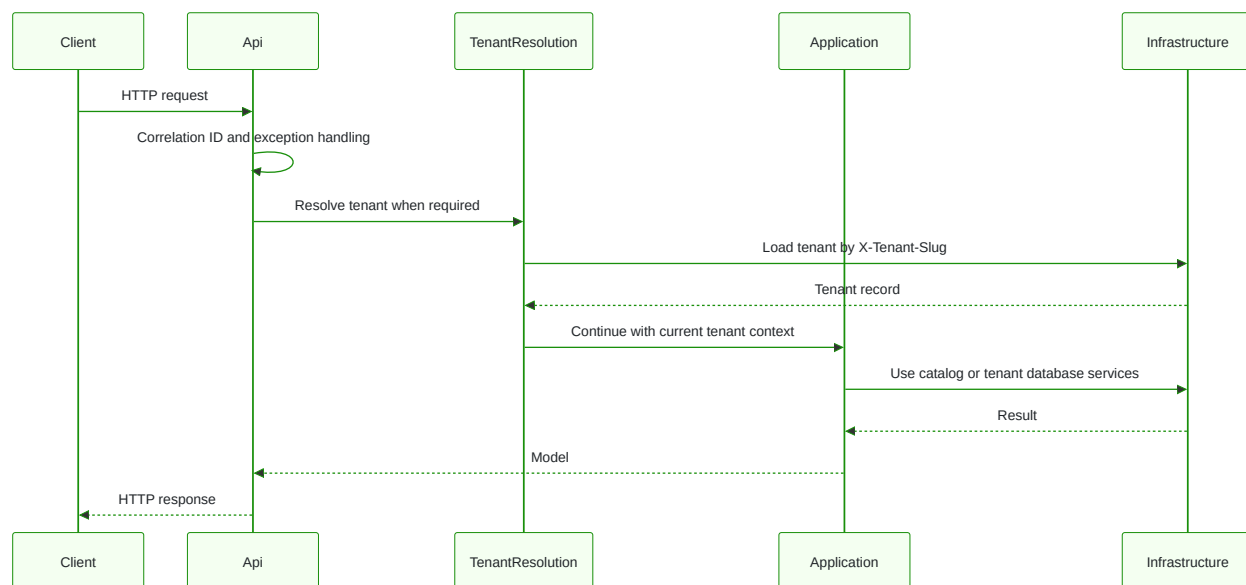
The infrastructure project owns external resources:

- `CatalogDbContext` stores tenant records.
- `OrganizorDbContext` connects to the current tenant database after tenant resolution.
- `TenantRepository` implements catalog tenant persistence.
- `ConnectionStringBuilder` builds PostgreSQL connection strings from configuration.
- `TenantDatabaseManager` creates and drops tenant databases.
- `TenantMigrator` applies EF Core migrations to tenant databases.
- `TenantProvisioningWorkflow` creates a tenant database and applies migrations during onboarding.
- `TenantJobScheduler` schedules tenant-scoped Quartz jobs from catalog tenant records.
- `TenantAwareJobFactory` creates Quartz jobs through dependency injection.
- `SyncTenantJob` runs recurring tenant synchronization work with `ITenantContext` populated from Quartz job data.

# Background Jobs

Quartz is registered by the API layer and backed by persistent PostgreSQL storage in the catalog database. On startup, the application schedules a recurring synchronization job for each catalog tenant. Each job carries the tenant ID and slug in Quartz job data, then sets the tenant context before running tenant database work.

## Request Pipeline



# Tenancy

Organizor uses catalog-driven, database-per-tenant tenancy.

## Tenant Catalog

The catalog database stores tenant metadata through `CatalogDbContext`. Each tenant includes:

- `Id` ;
- `Name` ;
- `Slug` ;
- `DatabaseName` ;
- `CreatedAt` ;
- `Status` ;
- `FailureReason` ;
- soft-delete metadata.

The database name convention is `organizor_tenant_{slug}`. Slugs are normalized to lowercase before they are stored or used for lookups.

Normal catalog reads exclude soft-deleted tenants. Repository methods can include soft-deleted records when an internal operation explicitly asks for them.

## Tenant Statuses

| Status    | Meaning                                                         |
|-----------|-----------------------------------------------------------------|
| Pending   | Tenant exists in the catalog but provisioning is not complete.  |
| Active    | Tenant provisioning completed successfully.                     |
| Upgrading | Reserved for upgrade lifecycle state.                           |
| Failed    | Provisioning failed or an unexpected onboarding error occurred. |
| Deleted   | Tenant has been soft-deleted.                                   |

# Tenant Resolution

`TenantResolutionMiddleware` resolves tenant context for tenant-scoped routes.

It skips platform endpoints such as:

- `/api/admin`;
- `/api/v1/tenants`;
- `/health`;
- `/scalar`;
- `/openapi`;
- root and common static paths.

For tenant-scoped requests, clients must send:

```
X-Tenant-Slug: acme
```

The middleware loads the tenant from the catalog and stores `TenantInfo` in `ITenantContext`. If the header is missing, the response is `400 Bad Request`. If the slug does not match a catalog tenant, the response is `404 Not Found`.

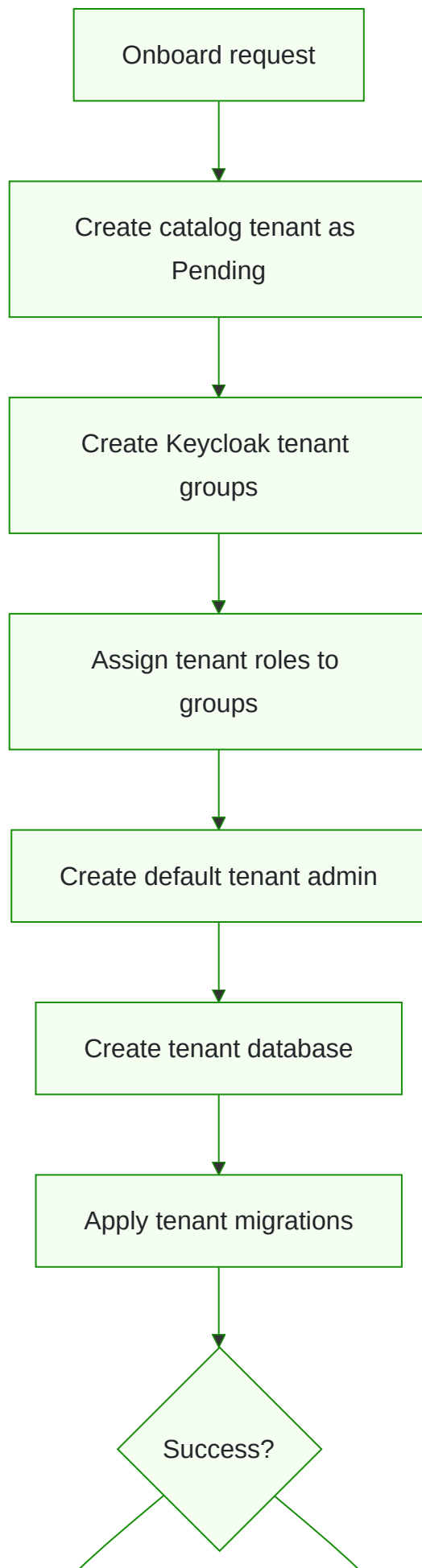
## Tenant Database Resolution

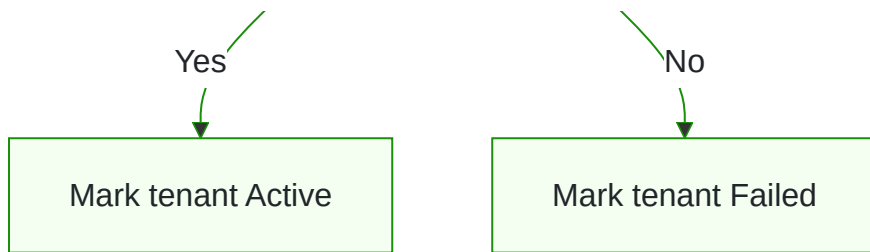
`OrganizorDbContext` uses a fallback in-memory database when no tenant has been resolved. This supports startup, Scalar, migrations, and design-time tooling.

When a tenant is resolved, `PerTenantDbResolver` loads the catalog tenant, builds a PostgreSQL connection string for that tenant database, and `OrganizorDbContext` uses that database.

## Tenant Onboarding Flow







Hard deletion is intentionally guarded: the tenant must be soft-deleted before `TenantLifecycleService` runs the hard-delete workflow. That workflow removes the Keycloak tenant group and default admin user, drops the tenant database, and removes the catalog record.



# API

The public API currently exposes tenant catalog endpoints. These endpoints are platform/catalog routes, so tenant resolution does not require `X-Tenant-Slug` for them. They are still authenticated:

- `POST /api/v1/tenants` requires `organizer-admin`.
- `GET /api/v1/tenants` and `GET /api/v1/tenants/{slug}` require `organizer-admin`, `tenant-admin`, or `tenant-user`.

Tenant-scoped application endpoints should include the `X-Tenant-Slug` header unless the path is explicitly skipped by tenant resolution.

Internal tenant administration endpoints are documented separately in [Internal Operations](#).

## Tenant Catalog

### Create Tenant

```
POST /api/v1/tenants
Authorization: Bearer <organizer-admin-token>
Content-Type: application/json

{
  "name": "Acme",
  "slug": "acme"
}
```

Creates a catalog tenant record with `Pending` status and returns tenant metadata. This route does not provision Keycloak groups, a default tenant admin, or a tenant database. Use `POST /api/admin/tenants/onboard` for the full onboarding workflow.

Example response:

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "name": "Acme",
```

```
"slug": "acme",
"databaseName": "organizer_tenant_acme",
"createdAt": "2026-05-15T10:00:00Z"
}
```

Possible responses:

| Status                    | Meaning                                                                 |
|---------------------------|-------------------------------------------------------------------------|
| 200 OK                    | Tenant catalog record was created.                                      |
| 401 Unauthorized          | Missing or invalid bearer token.                                        |
| 403 Forbidden             | Token does not satisfy the <code>organizer-admin</code> role policy.    |
| 409 Conflict              | Duplicate tenant slug.                                                  |
| 500 Internal Server Error | Unexpected server error. See <a href="#">Errors and Observability</a> . |

## Get Tenant By Slug

```
GET /api/v1/tenants/acme
Authorization: Bearer <token>
```

Returns a tenant by slug, or `404 Not Found` when no tenant exists. Soft-deleted tenants are excluded.

## List Tenants

```
GET /api/v1/tenants
Authorization: Bearer <token>
```

Returns all catalog tenants visible through the tenant repository. Soft-deleted tenants are excluded.

## Platform Endpoints

| Endpoint                      | Purpose                              |
|-------------------------------|--------------------------------------|
| <code>/health</code>          | Public health check endpoint.        |
| <code>/openapi/v1.json</code> | OpenAPI document in development.     |
| <code>/scalar</code>          | Scalar API reference in development. |

## Local API Reference

When the API runs in development:

- OpenAPI JSON is available at `/openapi/v1.json`.
- Scalar is available at `/scalar`.

The DocFX API navigation tab contains the generated C# reference for controllers, DTOs, services, interfaces, middleware, and persistence types.

# Database Provisioning

Organizor uses one catalog database and one PostgreSQL database per tenant.

## Catalog Database

`CatalogDbContext` stores tenant records in the `tenants` table.

Important constraints:

| Field                       | Constraint                                 |
|-----------------------------|--------------------------------------------|
| <code>Name</code>           | Required, max length 200.                  |
| <code>Slug</code>           | Required, max length 100, unique index.    |
| <code>DatabaseName</code>   | Required, max length 200.                  |
| <code>IsDeleted</code>      | Required, defaults to <code>false</code> . |
| <code>DeletionReason</code> | Optional, max length 500.                  |

On startup, the API checks whether the catalog database exists, creates it when needed, and applies catalog migrations.

The full tenant onboarding workflow is documented in [Internal Operations](#). Database provisioning is one part of that workflow; Keycloak tenant access is provisioned before the tenant database is created.

## Tenant Database Naming

Tenant databases use:

```
organizor_tenant_{slug}
```

Example:

organizer\_tenant\_acme

Slugs are normalized to lowercase before tenant records are written.

## PostgreSQL Requirements

The configured PostgreSQL user must be able to:

- connect to the admin `postgres` database;
- create tenant databases;
- drop tenant databases during hard deletion;
- create extensions in tenant databases.

Tenant database creation currently uses:

```
CREATE DATABASE "{databaseName}" OWNER = postgres ENCODING = 'UTF8';
```

Each tenant database is initialized with:

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";  
CREATE EXTENSION IF NOT EXISTS "citext";
```

Hard deletion uses:

```
DROP DATABASE IF EXISTS "{databaseName}" WITH (FORCE);
```

## Migration Flow

`TenantMigrator` builds a tenant-specific connection string, checks pending migrations on `OrganizerDbContext`, applies them, and returns:

| Field                 | Meaning                       |
|-----------------------|-------------------------------|
| <code>Database</code> | Tenant database name.         |
| <code>Success</code>  | Whether migrations completed. |

| Field             | Meaning                                       |
|-------------------|-----------------------------------------------|
| AppliedMigrations | Count of migrations pending before execution. |
| Duration          | Migration runtime.                            |
| Error             | Error message when migration fails.           |

## Onboarding and Cleanup

During `POST /api/admin/tenants/onboard`, the database steps are:

1. create the tenant database named `organizer_tenant_{slug}`;
2. initialize required PostgreSQL extensions;
3. apply pending `OrganizerDbContext` migrations;
4. return success so the tenant can be marked `Active`.

If the migration result fails or onboarding throws unexpectedly, the catalog tenant is marked `Failed`.

During hard deletion, the tenant must already be soft-deleted. The hard-delete workflow drops the tenant database with `DROP DATABASE IF EXISTS ... WITH (FORCE)` after Keycloak cleanup and then removes the catalog record.

# Keycloak Setup

This document describes the Keycloak configuration for the Organizer development environment. It includes realm setup, API/UI clients, tenant groups, claim mapping, user/role setup, and the admin-client permissions used by tenant onboarding.

## Realm Setup

- Open Keycloak Admin Console.
- Realm dropdown -> **Create Realm**.
- Name: `organizer-dev`.
- Save.

## API Client ( `organizer-api` )

Settings:

- Client type: OpenID Connect
- Client ID: `organizer-api`
- Name: Organizer API

Required values:

- Client authentication: **ON**
- Authorization: OFF
- Standard flow: OFF
- Direct access grants: **ON**
- Implicit flow: OFF
- Service accounts roles: **ON**

Credentials:

- Open the **Credentials** tab.
- Copy the **Client Secret**.
- Store it in development tooling that requests tokens.

The API currently validates bearer tokens only; it does not read the client secret from application configuration. The client secret is only needed by tooling or services that request tokens with the `client_credentials` flow.

The infrastructure Keycloak admin client also uses this client secret during tenant onboarding. Configure these application settings from the same Keycloak client:

```
{
  "Keycloak": {
    "Authority": "http://localhost:8080/realms/organizer-dev",
    "BaseUrl": "http://localhost:8080",
    "Realm": "organizer-dev",
    "AdminClientId": "organizer-api",
    "AdminClientSecret": "<client-secret>"
  }
}
```

For tenant onboarding and hard deletion, the `organizer-api` service account must be allowed to use the Keycloak Admin REST API. Assign the required `realm-management` client roles for:

- reading groups and users;
- creating and deleting tenant groups;
- assigning realm roles to groups;
- creating the default tenant admin user;
- resetting the default tenant admin password;
- adding users to groups;
- deleting the default tenant admin user during hard deletion.

## UI Client ( `organizer-ui` )

Settings:

- Client type: OpenID Connect
- Client ID: `organizer-ui`
- Name: Organizer UI

Required values:



- Client authentication: OFF
- Authorization: OFF
- Standard flow: **ON**
- Direct access grants: OFF
- Implicit flow: OFF
- Service accounts roles: OFF

Development redirect URIs:

- `https://localhost:5001/*`
- `https://localhost:5002/*`
- `http://localhost:5001/*`
- `http://localhost:5002/*`

Web origins:

- `*`

## Tenant Groups

Tenant onboarding creates the group structure automatically when `POST /api/admin/tenants/onboard` is used. Manual setup is only needed for bootstrap or testing.

Group structure:

- `/tenants`
- `/tenants/acme`
- `/tenants/acme/users`
- `/tenants/acme/admins`

Each tenant has:

- `/users` group
- `/admins` group

On onboarding, Organizer assigns:

- `tenant-admin` to `/tenants/{slug}/admins`;

- `tenant-user` to `/tenants/{slug}/users`.

It also creates a default tenant admin user:

- Username: `{slug}-admin`
- Email: `{slug}-admin@organizor.local`
- Temporary password: `ChangeMe123!`
- Group: `/tenants/{slug}/admins`

During hard deletion, Organizor removes the tenant group `/tenants/{slug}` and deletes the default `{slug}-admin` user when it exists.

## Tenant Membership Claim Mapper

Location:

- Clients -> `organizor-api`
- Client scopes -> `organizor-api-dedicated`
- Mappers -> Create

Values:

- Name: `tenant-membership`
- Mapper type: **Group Membership**
- Token Claim Name: `tenant`
- Full group path: **ON**
- Add to ID token: ON
- Add to access token: ON
- Add to userinfo: ON

## User Setup

Create these realm roles before assigning users:

- `organizor-admin`
- `tenant-admin`
- `tenant-user`

Platform admin:

- Username: `admin@organizor.dev`
- Password: `Admin123!`
- Assign realm role: `organizor-admin`
- No tenant groups assigned

Tenant admin:

- Username: `admin@acme.dev`
- Password: `Admin123!`
- Assign realm role: `tenant-admin`
- Assign group: `/tenants/acme/admins`

Tenant user:

- Username: `user@acme.dev`
- Password: `Admin123!`
- Assign realm role: `tenant-user`
- Assign group: `/tenants/acme/users`

## Token Testing

Token endpoint:

- `http://localhost:8080/realms/organizor-dev/protocol/openid-connect/token`

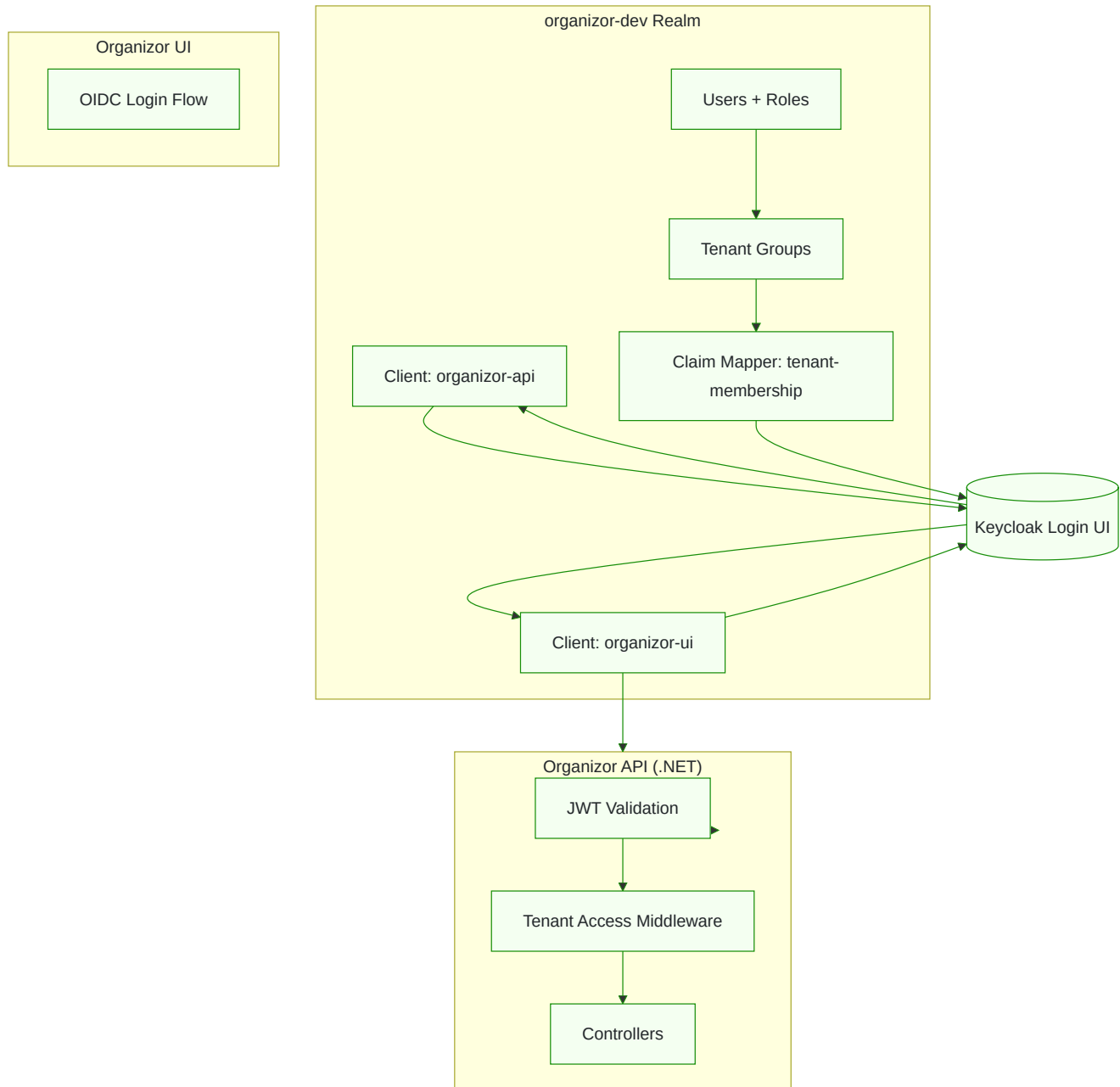
Expected role claim examples:

```
{
  "realm_access": {
    "roles": ["organizor-admin"]
  },
  "resource_access": {
    "organizor-api": {
      "roles": ["tenant-user"]
    }
  }
}
```

Example tenant claim:

```
"tenant": ["/tenants/acme/admins"]
```

# Keycloak Architecture Diagram



# API Authentication

## Overview

This document explains how the Organizer API authenticates requests using Keycloak-issued JWT tokens.

## JWT Validation

The API validates tokens using the Keycloak realm:

- Authority: `http://localhost:8080/realms/organizer-dev`
- Audience validation: disabled for the current development setup
- Token type: Bearer

The authentication middleware:

- validates the token signature;
- validates the issuer;
- validates token expiration;
- populates `HttpContext.User`.

Audience validation is intentionally disabled in code while the Keycloak client and token shape are still being finalized. Once the API has a stable audience claim, enable audience validation and document the expected value here.

## Authentication Flow

1. Frontend logs in via Keycloak.
2. Keycloak issues a JWT.
3. Frontend sends `Authorization: Bearer <token>`.
4. Frontend also sends `X-Tenant-Slug: acme` for tenant-scoped routes.
5. API validates the JWT.
6. `[Authorize]` attributes enforce authentication where applied.

# Service Accounts

The `organizer-api` client can use the `client_credentials` flow for internal API-to-API communication. Service account tokens are identified by the `client_id` claim:

```
"client_id": "organizer-api"
```

Service accounts bypass tenant access checks.

## WhoAmI Endpoint

Example response:

```
{
  "user": "admin@acme.dev",
  "tenant": null,
  "tenantClaims": [
    "/tenants/acme/admins"
  ],
  "roles": [
    "tenant-admin",
    "tenant-user"
  ],
  "claims": [
    { "type": "sub", "value": "..." },
    { "type": "tenant", "value": "/tenants/acme/admins" },
    { "type": "role", "value": "tenant-admin" }
  ]
}
```

## Error Codes

- 401 Unauthorized -> Missing or invalid token.
- 403 Forbidden -> Token is valid, but the user is not allowed.
- 400 Bad Request -> Missing or invalid tenant header on tenant-scoped routes.

## Test Request

GET http://localhost:5100/api/auth/whoami

**Authorization:** Bearer <token>

# Role-Based Authorization

## Overview

The API authorizes requests with ASP.NET Core policies backed by Keycloak roles. Authentication validates the JWT first, tenant access middleware enforces tenant membership next, and endpoint authorization policies run after that.

## Roles

| Role                         | Purpose                                                                                    |
|------------------------------|--------------------------------------------------------------------------------------------|
| <code>organizer-admin</code> | Platform administrator. Can manage tenants and call admin operations.                      |
| <code>tenant-admin</code>    | Tenant administrator. Can use tenant-aware application endpoints and tenant catalog reads. |
| <code>tenant-user</code>     | Tenant member. Can use shared tenant-aware application endpoints and tenant catalog reads. |

## JWT Role Sources

The API maps these Keycloak claim shapes to ASP.NET Core role claims:

```
{
  "realm_access": {
    "roles": ["organizer-admin"]
  },
  "resource_access": {
    "organizer-api": {
      "roles": ["tenant-user"]
    }
  }
}
```



Only client roles under `resource_access.organizor-api.roles` are promoted. Roles for other clients, such as `account`, are ignored for API authorization.

## Policies

| Policy                                 | Accepted roles                                                                      |
|----------------------------------------|-------------------------------------------------------------------------------------|
| <code>RequireOrganizorAdminRole</code> | <code>organizor-admin</code>                                                        |
| <code>RequireTenantUserRole</code>     | <code>organizor-admin</code> , <code>tenant-admin</code> , <code>tenant-user</code> |
| <code>RequireTenantAdminRole</code>    | <code>organizor-admin</code> , <code>tenant-admin</code>                            |

Use policies for new endpoints unless the endpoint is intentionally authentication-only.

```
[Authorize(Policy = AuthorizationPolicies.RequireOrganizorAdminRole)]
```

## Current Endpoint Enforcement

| Endpoint area                           | Policy                                 |
|-----------------------------------------|----------------------------------------|
| <code>/api/admin/tenants/*</code>       | <code>RequireOrganizorAdminRole</code> |
| <code>POST /api/v1/tenants</code>       | <code>RequireOrganizorAdminRole</code> |
| <code>GET /api/v1/tenants</code>        | <code>RequireTenantUserRole</code>     |
| <code>GET /api/v1/tenants/{slug}</code> | <code>RequireTenantUserRole</code>     |
| <code>GET /api/auth/whoami</code>       | authenticated user only                |

## Tenant Interaction

Role checks do not replace tenant checks. For tenant-scoped routes, a user must pass both:

- Tenant authorization in `TenantAccessMiddleware`.
- Endpoint role or policy authorization in ASP.NET Core.

This means a `tenant-user` token for `acme` is still denied when it sends `X-Tenant-Slug: contoso`.

## Assigning Roles in Keycloak

Realm roles are assigned from the user details screen:

1. Open Keycloak Admin Console.
2. Select the `organizer-dev` realm.
3. Open Users and select the user.
4. Open Role mapping.
5. Assign one of `organizer-admin`, `tenant-admin`, or `tenant-user`.

Client roles can also be assigned under the `organizer-api` client. If client roles are used, verify they appear in `resource_access.organizer-api.roles`.

## Test Matrix

| Scenario                                                           | Expected      |
|--------------------------------------------------------------------|---------------|
| <code>organizer-admin</code> calls <code>/api/admin/tenants</code> | Allowed       |
| <code>tenant-user</code> calls <code>/api/admin/tenants</code>     | 403 Forbidden |
| <code>tenant-admin</code> calls <code>GET /api/v1/tenants</code>   | Allowed       |
| <code>tenant-user</code> calls <code>GET /api/v1/tenants</code>    | Allowed       |
| Token has no Organizer role and calls protected endpoint           | 403 Forbidden |
| Valid role with wrong tenant slug on tenant-scoped route           | 403 Forbidden |

# Tenant Authorization

## Overview

Tenant-scoped API routes require both a valid JWT and an `X-Tenant-Slug` header. The API resolves the header to a catalog tenant, validates that the JWT is allowed to use that tenant, then lets controller authorization policies run.

## Tenant Resolution

Tenant-scoped routes must include:

```
X-Tenant-Slug: acme
```

The tenant resolution middleware skips platform and infrastructure routes, including:

- `/api/admin`
- `/api/v1/tenants`
- `/api/auth`
- `/health`
- `/openapi`
- `/scalar`
- `/favicon`
- `/robots`

Resolution steps:

1. Middleware reads `X-Tenant-Slug`.
2. Middleware looks up the tenant in the catalog database.
3. Middleware sets `tenantContext.Current`.
4. Middleware logs tenant metadata.

## JWT Tenant Claim

Tenant membership is represented by one or more `tenant` claims. The middleware accepts either direct slugs or Keycloak-style paths:

```
{
  "tenant": ["/tenants/acme/admins", "/tenants/contoso/users"]
}
```

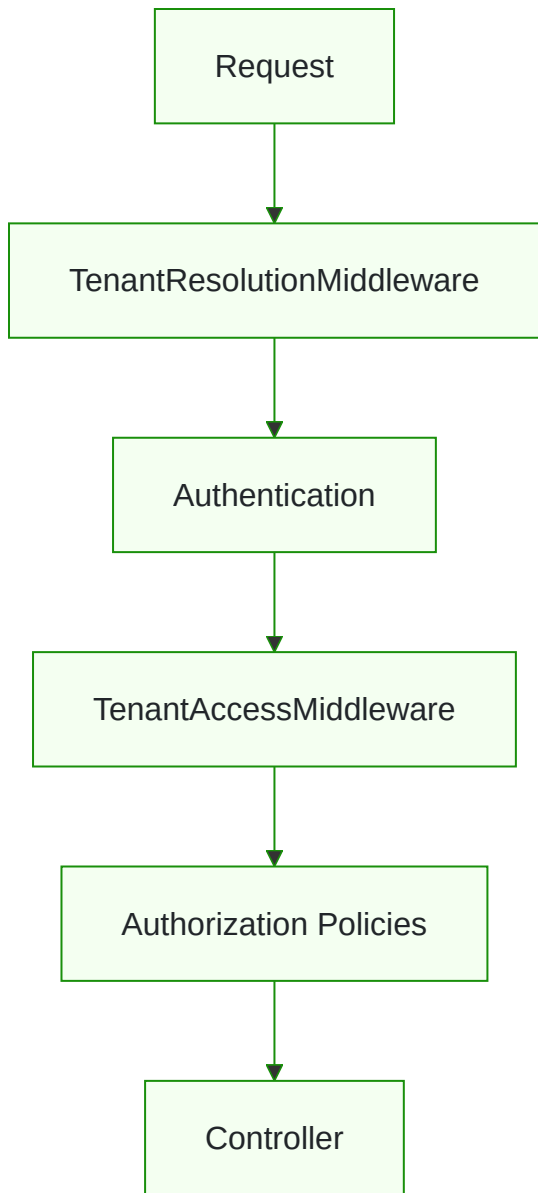
The slug is extracted from the segment after `/tenants/`. For `/tenants/acme/admins`, the effective tenant slug is `acme`.

## Enforcement Rules

For tenant-scoped routes:

- Missing or invalid `Authorization` returns `401 Unauthorized` when the endpoint requires `[Authorize]`.
- Missing `X-Tenant-Slug` returns `400 Bad Request`.
- Unknown `X-Tenant-Slug` returns `404 Not Found`.
- A JWT with tenant claims must include the same slug as `X-Tenant-Slug`.
- A JWT tenant/header mismatch returns `403 Forbidden`.
- A JWT without tenant claims is treated as a platform superadmin token and bypasses tenant membership checks.
- The `organizer-api` service account, identified by `client_id = organizer-api`, bypasses tenant membership checks.
- Matching tenant JWTs are still checked through `ITenantAccessService.UserHasAccessAsync(userId, tenantId)`.
- If `ITenantAccessService` denies access, the API returns `403 Forbidden`.

## Middleware Chain



# HTTP Test Matrix

| Scenario                                                      | Authorization | X-Tenant-Slug        | Expected                                            |
|---------------------------------------------------------------|---------------|----------------------|-----------------------------------------------------|
| Tenant token for <code>acme</code> calls <code>acme</code>    | tenant JWT    | <code>acme</code>    | Allowed if <code>ITenantAccessService</code> allows |
| Tenant token for <code>acme</code> calls <code>contoso</code> | tenant JWT    | <code>contoso</code> | 403 Forbidden                                       |

| Scenario                                               | Authorization                          | X-Tenant-Slug  | Expected                      |
|--------------------------------------------------------|----------------------------------------|----------------|-------------------------------|
| Superadmin token has no tenant claim                   | superadmin JWT                         | any valid slug | Allowed                       |
| Service account token                                  | <code>client_id = organizer-api</code> | any valid slug | Allowed                       |
| Missing tenant header on tenant route                  | valid JWT                              | omitted        | <code>400 Bad Request</code>  |
| Unknown tenant header on tenant route                  | valid JWT                              | unknown slug   | <code>404 Not Found</code>    |
| Missing token on authorized route                      | omitted                                | valid slug     | <code>401 Unauthorized</code> |
| <code>ITenantAccessService</code> denies matching user | tenant JWT                             | matching slug  | <code>403 Forbidden</code>    |

## Sample Requests

Tenant-scoped request:

```
GET http://localhost:5100/api/some-tenant-scoped-route
Authorization: Bearer <tenant-token>
X-Tenant-Slug: acme
Accept: application/json
```

Cross-tenant denial:

```
GET http://localhost:5100/api/some-tenant-scoped-route
Authorization: Bearer <token-with-tenant-acme>
X-Tenant-Slug: contoso
Accept: application/json
```

Superadmin bypass:

```
GET http://localhost:5100/api/some-tenant-scoped-route
Authorization: Bearer <token-without-tenant-claim>
X-Tenant-Slug: acme
Accept: application/json
```

# Configuration

Organizor reads standard ASP.NET Core configuration from `appsettings.json`, environment-specific files, user secrets, and environment variables.

## Catalog Connection String

`CatalogDbContext` expects a connection string named `Catalog`.

Example shape:

```
{
  "ConnectionStrings": {
    "Catalog":
      "Host=localhost;Port=5432;Database=organizor_catalog;Username=postgres;Password=postgres"
  }
}
```

## Tenant PostgreSQL Settings

`ConnectionStringBuilder` builds tenant and admin connection strings from the `Postgres` section.

```
{
  "Postgres": {
    "Host": "localhost",
    "Port": "5432",
    "User": "postgres",
    "Password": "postgres"
  }
}
```

The admin connection uses the `postgres` database. Tenant connections use the tenant database name stored in the catalog.

## Keycloak



The API requires Keycloak settings for two separate concerns:

- JWT validation uses `Keycloak:Authority`.
- Tenant onboarding uses the Keycloak admin client settings to create tenant groups, assign roles, and create the default tenant admin user.

Example development shape:

```
{
  "Keycloak": {
    "Authority": "http://localhost:8080/realms/organizer-dev",
    "BaseUrl": "http://localhost:8080",
    "Realm": "organizer-dev",
    "AdminClientId": "organizer-api",
    "AdminClientSecret": "<client-secret>"
  }
}
```

`Authority` must match the token issuer exactly. In development, HTTPS metadata validation is disabled; outside development, Keycloak should be served over HTTPS.

Store client secrets in user secrets, environment variables, or deployment secrets rather than committing them to source.

## Quartz

Quartz uses the `Catalog` connection string for its persistent store. The same catalog database stores tenant records and Quartz scheduler tables.

The API registration configures:

- PostgreSQL-backed Quartz persistence;
- Newtonsoft JSON serialization for persisted job data;
- clustering support;
- hosted service startup for Quartz background processing.

Tenant job scheduling is not enabled on application startup yet. `TenantJobScheduler` is available for future real tenant jobs, but the startup scheduling block is currently commented out until those jobs exist.

# Logging

Serilog is configured in `appsettings.json`.

Current sinks:

- console;
- rolling file logs under `logs/log-.txt`.

Log output includes tenant and request correlation properties when available:

- `Id`;
- `Slug`;
- `DatabaseName`;
- `CorrelationId`.

## CORS

The API selects a CORS policy by environment:

- `DevCors` in development allows any origin, header, and method.
- `ProdCors` outside development must be configured with explicit allowed origins before browser clients can call the API.

The current production policy has no origins configured. Treat that as a deployment checklist item, not a complete production setting.

## OpenAPI and Scalar

OpenAPI and Scalar endpoints are mapped only in development. This keeps the production surface smaller while retaining local API exploration during development.

# Localization

Organizor provides multilingual support for error responses using a simple, predictable, API-friendly localization pipeline.

## Supported Languages

Organizor currently supports:

- `en` - English (fallback)
- `nl` - Dutch

Additional languages can be added by creating new resource files in the `Code7Dev.Organizor.Api/Resources` folder.

## How Localization Works

Localization is driven by the standard `Accept-Language` HTTP header.

Example:

```
Accept-Language: nl
```

The API resolves the culture for the request and uses it to translate error messages. If a translation is missing, the system falls back to English (`en`).

## Key Principles

- Application services return error codes only, such as `TENANT_NOT_FOUND`.
- The API middleware maps those codes to localized resource strings.
- No hardcoded user-facing messages exist in the domain or application layers.
- Localization is deterministic and client-controlled.

## Culture Resolution

Organizor uses the following resolution order:

- `Accept-Language` header.
- Fallback to English ( `en` ).

This keeps behavior predictable and client-controlled.

## Error Code Translation

Each error code corresponds to a resource key.

Examples:

- `TENANT_NOT_FOUND`
- `TENANT_ALREADY_EXISTS`
- `UNEXPECTED_ERROR`

These keys exist in:

- `Code7Dev.Organizor.Api/Resources/Shared.en.resx`
- `Code7Dev.Organizor.Api/Resources/Shared.nl.resx`

The middleware retrieves the localized string and injects it into the RFC 7807 problem response.

## Example Request and Response

Request:

```
GET /api/v1/tenants/unknown
Authorization: Bearer <token>
Accept-Language: nl
```

Response:

```
{
  "type": "https://httpstatuses.com/404",
  "title": "Tenant niet gevonden",
  "status": 404,
  "error": "TENANT_NOT_FOUND",
```

```
"correlationId": "abc123"  
}
```

The error code remains stable and language-independent. The title is localized based on the request culture.

## Adding New Translations

To add a new language:

1. Create a new `.resx` file, for example `Shared.fr.resx`.
2. Add the same error keys as in the English file.
3. Provide translated values.
4. Register the culture in `AddApiLocalization`.
5. Restart the API.

# Errors and Observability

## Correlation IDs

Every request passes through `CorrelationIdMiddleware`.

Clients can provide a correlation ID:

```
X-Correlation-ID: request-123
```

If the header is missing, the API generates a GUID. The final value is returned in the response header:

```
X-Correlation-ID: 2ad71e7a-13cb-4ef8-a4c7-f3a37ad9dc71
```

The same value is pushed into Serilog context as `CorrelationId`.

## Error Shape

Unhandled exceptions are returned as RFC 7807-style problem details:

```
{
  "type": "https://httpstatuses.com/500",
  "title": "An unexpected error occurred.",
  "status": 500,
  "detail": "Exception message",
  "instance": "/api/v1/tenants",
  "correlationId": "2ad71e7a-13cb-4ef8-a4c7-f3a37ad9dc71"
}
```

The response content type is:

```
application/problem+json
```

### NOTE

The current middleware includes exception messages in `detail`. Treat that as useful during development, but review it before exposing production environments.

## Tenant Resolution Errors

Tenant-scoped requests require:

`X-Tenant-Slug`: acme

Missing tenant slug returns `400 Bad Request` with a plain text response. Unknown tenant slug returns `404 Not Found` with a plain text response.

## Log Context

Serilog output is configured to include:

| Property                   | Source                                    |
|----------------------------|-------------------------------------------|
| <code>CorrelationId</code> | Request header or generated GUID.         |
| <code>Id</code>            | Resolved tenant ID.                       |
| <code>Slug</code>          | Resolved tenant slug.                     |
| <code>DatabaseName</code>  | Tenant database selection when available. |
| <code>SourceContext</code> | Logger source in file logs.               |

# Internal Operations

This page documents internal routes and workflows for developers and operators.

## Onboard Tenant

```
POST /api/admin/tenants/onboard
Authorization: Bearer <organizer-admin-token>
Content-Type: application/json
```

```
{
  "name": "Acme",
  "slug": "acme"
}
```

The onboarding flow:

1. normalizes the slug to lowercase;
2. creates a catalog tenant with `Pending` status;
3. provisions Keycloak tenant groups under `/tenants/{slug}`;
4. assigns `tenant-admin` to `/tenants/{slug}/admins` and `tenant-user` to `/tenants/{slug}/users`;
5. creates a default tenant admin user named `{slug}-admin` with a temporary password of `ChangeMe123!`;
6. creates the tenant database using `organizer_tenant_{slug}`;
7. initializes PostgreSQL extensions;
8. applies tenant migrations;
9. marks the tenant `Active` on success or `Failed` on provisioning failure.

Example response:

```
{
  "id": "11111111-1111-1111-1111-111111111111",
  "name": "Acme",
  "slug": "acme",
  "databaseName": "organizer_tenant_acme"
}
```



# Upgrade Tenant

```
POST /api/admin/tenants/{id}/upgrade
Authorization: Bearer <organizer-admin-token>
```

Runs pending migrations for the tenant database.

Example response:

```
{
  "database": "organizer_tenant_acme",
  "success": true,
  "appliedMigrations": 2,
  "duration": "00:00:01.2340000",
  "error": null
}
```

# Soft Delete Tenant

```
DELETE /api/admin/tenants/{id}
Authorization: Bearer <organizer-admin-token>
Content-Type: application/json
```

```
{
  "reason": "Customer cancellation"
}
```

Soft deletion marks the tenant as deleted, stores deletion metadata, and sets status to `Deleted`. The tenant database is not dropped.

# Hard Delete Tenant

```
POST /api/admin/tenants/{id}/hard-delete
Authorization: Bearer <organizer-admin-token>
```

Hard deletion requires the tenant to already be soft-deleted. It:

1. deletes the default Keycloak admin user named `{slug}-admin` when it exists;

2. deletes the Keycloak tenant group `/tenants/{slug}` when it exists;
3. drops the tenant database;
4. removes the catalog record.

Keycloak cleanup tolerates missing groups or users so a partially-provisioned tenant can still be cleaned up.

## Scheduled Tenant Jobs

Organizor is wired for Quartz recurring background work, but tenant job scheduling is not currently enabled on startup. `Program.cs` contains a commented-out scheduling block for future use.

The scheduler:

1. loads all tenants from the catalog database;
2. creates a durable `SyncTenantJob` with job key `sync-{tenantId}` in the `tenant-jobs` group;
3. stores `TenantId` and `TenantSlug` in the Quartz job data map;
4. creates a trigger named `sync-{tenantId}-trigger` in the `tenant-triggers` group;
5. schedules the job when it is new, or replaces and reschedules it when it already exists.

Current trigger cadence:

```
0/10 * * * * ?
```

That Quartz cron expression would run the tenant synchronization job every 10 seconds when scheduling is enabled.

`SyncTenantJob` reads the tenant data from the job data map, sets `ITenantContext`, and then runs tenant-scoped work through `OrganizorDbContext`. The current implementation logs execution and contains placeholder synchronization work.

Quartz persistence is stored in the catalog database and uses the PostgreSQL tables added by the `AddQuartzTables` migration. The API configures Quartz with the `Catalog` connection string, JSON serialization, and clustering enabled.

# Documentation Deployment

The repository contains a GitHub Actions workflow at `.github/workflows/docfx-deploy.yml`.

On pushes to `master`, it:

1. checks out this repository;
2. installs DocFX;
3. builds `docs/docfx.json`;
4. checks out the public documentation repository;
5. copies `docs/_site` into that repository;
6. commits and pushes the generated documentation.

The workflow uses the `DOCS_DEPLOY_KEY` secret to push to the public docs repository.

## Sitemap and Crawlers

DocFX generates `docs/_site/sitemap.xml` from the `build.sitemap.baseUrl` value in `docs/docfx.json`.

The documentation source includes `robots.txt`, which is copied into the generated site and points crawlers to the sitemap.